

Agile과 MSA 기반 의약품 추천 발주 서비스 보고서

MediBridge MVP 기획과 적용 방안

판교 4반 2조 | 개인 보고서

이름 _____

보고서 개요

MediBridge는 병원·약국의 의약품 탐색과 발주를 돕고, CSO의 거래처 관리 부담을 줄이기 위한 서비스이다. CSO는 제약사의 영업 업무를 지원하는 조직을 뜻한다. 이 보고서는 기존 구매 업무에 추천 기능을 연결하는 방법과 이를 단계적으로 개발하기 위한 Agile 및 MSA 적용 방안을 정리한다.

우선 구매 이력을 활용한 간단한 추천과 발주 흐름을 검증하는 것이 적절하다. 이후 구매 담당자의 의견을 반영해 추천 기준을 개선한다. 서비스는 제품, 발주, 결제, 추천의 책임에 따라 나누어 변경 범위를 줄이는 방향으로 설계한다.

MVP 범위와 검증 기준

MVP는 핵심 기능만 갖춘 초기 서비스이다. 실제 사용자가 추천을 활용하는지 확인하기 위해 로그인, 제품 조회, 발주를 먼저 연결하고, 결제 이벤트와 추천 기능을 다음 단계에서 검증한다.

구분	초기 범위	후속 확장
업무 흐름	로그인과 제품 조회, 발주 접수	결제 완료와 발주 확정, 추천 갱신
추천 방식	구매 약효군의 미구매 품목과 신규 거래처용 인기 품목	구매 주기와 재고를 반영한 재주문 제안
판단 기준	추천 클릭과 발주 전환, 추천 거절 사유	성과와 데이터 품질을 확인한 뒤 범위 확대

기대 효과는 제품을 찾는 시간과 반복 확인 업무를 줄이는 데 있다. 품질 감소나 매출 향상은 실제 운영 데이터를 통해 확인할 항목으로 둔다.

1 이해관계자 가치와 업무 문제

1.1 이해관계자별 요구

이해관계자	현재의 불편	제공할 가치
병원·약국 구매 담당자	영업 접점에 따라 얻는 정보가 다르고, 비슷한 품목을 비교하는 데 시간이 든다.	제품을 직접 비교하고 추천 이유를 확인한 뒤 발주한다.
CSO 운영사와 영업 담당자	거래처가 늘수록 방문과 응대 부담이 커지고, 제안이 개인 경험에 의존한다.	구매 이력을 바탕으로 상담을 준비하고 더 많은 거래처를 관리한다.
제약사 관리자	회사마다 제품 데이터 형식이 다르고 품목 정보가 자주 바뀐다.	등록 기준을 맞추고 변경된 정보를 빠르게 반영한다.
기술과 운영 담당자	기능이 강하게 연결되면 작은 수정도 전체 배포와 장애로 이어질 수 있다.	변경 범위를 줄이고 장애가 난 기능을 빠르게 찾는다.

1.2 신뢰를 유지하는 서비스 개선

구매 담당자가 쌓아 온 거래 경험과 신뢰는 서비스가 바뀌어도 유지해야 할 가치이다. 추천 이유가 불분명하거나 제품 정보가 틀리면 사용자는 결과를 다시 확인해야 한다. 따라서 추천 근거와 정보 갱신 시점을 보여 주고, 최종 발주는 담당자가 판단하도록 한다.

1.3 Agile을 통한 현장 의견 반영

Agile은 짧은 개발 주기인 스프린트마다 결과를 확인하고 개선하는 방식이다. 예를 들어 구매 담당자가 추천 품목의 가격 비교가 어렵다고 말하면, 다음 스프린트에서 비교에 필요한 정보를 먼저 보완한다. 기능을 계속 추가하는 것보다 실제 불편이 줄었는지 확인하는 일이 중요하다.

진행 상황은 할 일, 진행 중, 검증 완료로 나누어 공유한다. 검토 때는 사용자 의견, 남은 문제, 추가 인력과 작업 시간을 함께 살펴본다. 서비스의 목적은 유지하되 구현 순서와 범위는 결과에 따라 조정한다.

1.4 MSA를 통한 변경 범위 축소

MSA는 업무별로 서비스를 나누어 운영하는 구조이다. 추천 기준만 바뀌면 추천 서비스를 중심으로 수정할 수 있어 제품 조화와 발주에 미치는 영향을 줄일 수 있다. 이를 위해 데이터 관리 책임과 API 형식을 먼저 정하고, 추천 장애가 발주를 막지 않도록 호출 제한 시간과 오류 처리를 마련해야 한다.

2 추천 기능과 사용자 흐름

2.1 기존 발주 업무에 추천 연결

MediBridge는 구매 담당자의 기존 업무 순서에 추천 기능을 덧붙인다. 제품 탐색과 발주를 익숙하게 유지하면서, 필요한 후보를 더 쉽게 찾을 수 있도록 돕는 것이 목표이다. 다음 표는 결제와 추천 연동까지 포함한 목표 흐름이다.

단계	사용자 행동	시스템 처리	기대 효과
1	등록과 로그인	계정과 역할, 접근 권한 확인	업무별 접근 관리
2	제품 탐색	목록과 상세 정보, 약효군 필터 제공	비교 시간 단축
3	발주 신청	제품 확인과 중복 검증 후 PENDING 저장	발주 접수 확인
4	결제 진행	거래 식별자를 기록하고 완료 이벤트 발행	반복 확인 감소
5	발주 상태 확인	결제 완료를 받아 ACTIVE로 변경	진행 상태 공유
6	추천 후보 확인	구매 이력에 맞는 후보와 이유 제공	추가 품목 탐색 지원

2.2 설명하기 쉬운 추천 기준

구매 이력이 있는 거래처는 가장 자주 구매한 약효군을 기준으로 아직 구매하지 않은 품목을 최대 5개 추천받는다. 구매 이력이 없는 거래처에는 누적 발주량이 높은 인기 품목을 보여 준다. 초기에는 학습 모델 대신 이 규칙을 사용해 결과를 쉽게 설명하고 오류를 확인한다.

예를 들어 특정 약효군의 구매가 많은 약국에는 해당 약효군의 미구매 품목을 보여 주고, 그 이유를 함께 안내한다. 이는 비교할 후보를 좁히는 기능이다. 이미 구매한 품목을 다시 주문하는 기능이나 재고 부족을 예측하는 기능과는 구분한다.

2.3 추천의 장점과 보완 조건

이 방식은 데이터가 적어도 시작할 수 있고, 담당자가 추천 이유를 이해하기 쉽다는 장점이 있다. 반면 인기 품목이 모든 거래처에 적합한 것은 아니며, 구매 이력만으로 현재 재고를 알 수 없다. 추천할 후보가 없을 때의 안내와 판매 가능한 품목만 보여 주는 검증을 완료 기준에 포함한다.

약효군이 같다는 이유만으로 제품의 대체 가능성을 판단하지 않는다. 구매 추천은 품목 탐색을 돕는 범위로 두고, 제품별 정보와 사용 적합성은 별도로 확인하도록 한다.

2 추천 기능의 단계별 개발 계획

2.4 스프린트 운영과 자원 배분

구분	주요 작업	검토할 결과
스프린트 1	로그인, 제품 조회, 발주 생성과 조회를 연결한다.	처음부터 끝까지 기본 업무가 이어지는지 확인한다.
스프린트 2	결제, Kafka 이벤트, 추천 기능을 연결한다.	상태 변경과 추천 갱신, 중복 이벤트 처리를 확인한다.
후속 개선	추천 거절 사유와 사용 기록에 따라 우선순위를 바꾼다.	기능별 사용성과 운영 부담을 비교한다.

새 기능을 추가할 때는 개발 시간뿐 아니라 데이터 정리, 테스트, 배포, 운영 점검에 필요한 작업도 함께 계산한다. 예를 들어 제약사 데이터 형식이 바뀌면 제품 담당자가 변환 규칙을 수정하고, 추천 담당자는 추천 결과에 영향이 있는지 확인한다.

2.5 사용자 요구와 완료 기준

사용자 요구는 누가, 무엇을, 왜 필요로 하는지 적는다. 완료 기준은 화면이 만들어졌는지에 그치지 않고 정상 처리와 예외 상황까지 확인할 수 있어야 한다.

ID	사용자 요구	완료 기준	시점
US-04	구매 담당자는 필요한 품목을 비교하기 위해 약효군별로 제품을 찾는다.	필터 동작, 검색 결과 0건 안내, 상세 정보 표시	S1
US-06	구매 담당자는 재고를 보충하기 위해 제품을 발주한다.	제품 검증, PENDING 생성, 대기 중 중복 발주 차단	S1
US-08	운영 담당자는 수기 확인을 줄이기 위해 결제 후 발주 상태가 바뀌길 원한다.	완료 이벤트 수신, ACTIVE 변경, 같은 이벤트 재수신 시 중복 반영 방지	S2
US-11	구매 담당자는 비교 대상을 좁히기 위해 구매 이력과 관련된 미구매 품목을 찾는다.	구매 약효군 반영, 미구매 후보 최대 5개, 이력 없음과 후보 없음 처리	S2

2.6 피드백을 다음 작업으로 연결

스프린트 검토에서는 화면과 API 결과를 함께 보여 준다. 추천을 사용하지 않았다면 품목이 맞지 않았는지, 정보가 부족했는지, 발주 과정이 불편했는지 나누어 기록한다. 다음 작업 목록에는 가장 자주 나타난 불편과 운영 위험을 우선 반영한다.

진행도를 공유할 때는 Mermaid와 같은 텍스트 기반 다이어그램으로 서비스 흐름을 정리할 수 있다. 흐름도와 API 명세를 함께 갱신하면 담당자가 바뀌어도 기능 간 연결을 이해하기 쉽다.

3 아키텍처와 서비스 책임

웹 화면은 API Gateway를 통해 각 서비스에 접근한다. Gateway는 요청을 전달하는 진입점이고, Eureka는 서비스 이름과 주소를 연결하는 주소록이다. 인증 서버는 사용자 인증을 담당하며, 업무 서비스는 요청별 접근 권한을 확인한다.

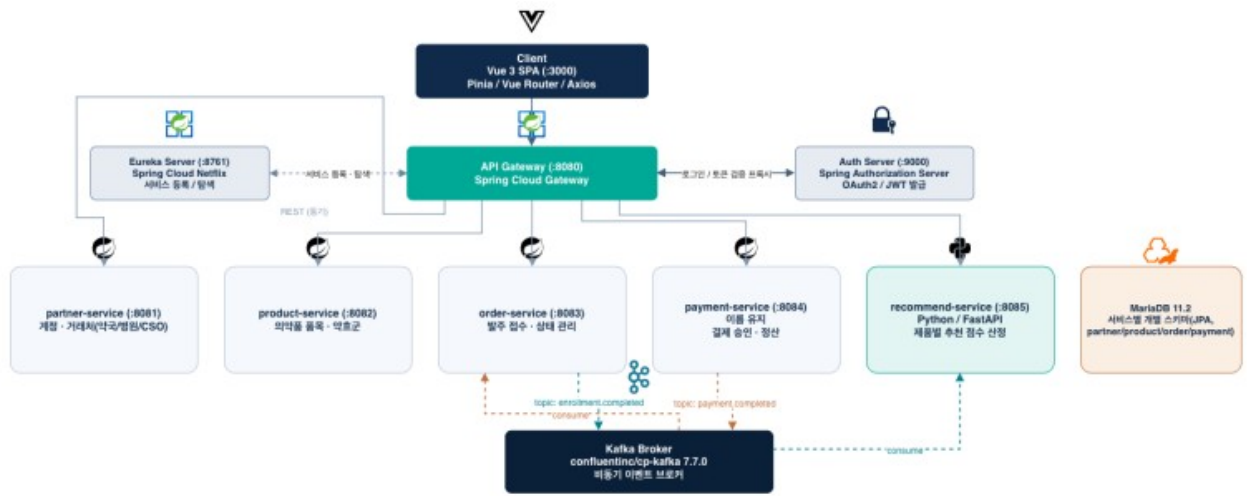


그림 1 MediBridge의 서비스 구성과 연결 구조

3.1 업무별 서비스 경계

서비스	관리 책임	분리했을 때의 장점
partner-service	사용자와 거래처 정보	회원 정보 변경 범위를 줄인다.
product-service	의약품 등록과 분류, 상세 정보	제품 형식 변경을 해당 서비스에서 처리한다.
order-service	발주 접수와 상태	추천 계산과 발주 처리를 분리한다.
payment-service	결제와 거래 기록	결제 오류를 따로 추적한다.
recommend-service	구매 이력을 이용한 후보 계산	추천 기준을 독립적으로 개선한다.

3.2 실습 구조의 활용

강의 서비스의 사용자, 강의, 수강 등록 구조를 거래처, 제품, 발주 업무에 대응시킬 수 있다. 다만 이름만 바꾸기보다 각 서비스가 관리할 데이터와 상태를 다시 정해야 한다. MariaDB를 함께 사용하더라도 서비스별 데이터 관리 범위를 구분하고, 다른 서비스의 테이블을 임의로 수정하지 않는 방향으로 설계한다.

Docker는 서비스별 실행 환경을 맞추는 데 활용한다. 서비스가 나뉘면 실행과 점검 대상도 늘어나므로, 초기에는 필요한 컨테이너만 구성하고 실행 방법과 포트 정보를 함께 관리한다.

3 업무 흐름과 검증 범위

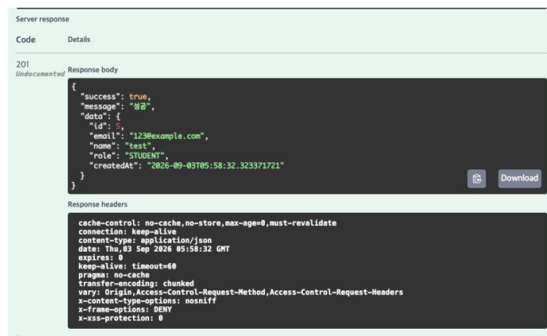
3.3 발주부터 추천 갱신까지

목표 흐름은 로그인 → 제품 조회 → 발주 접수(PENDING) → 결제 완료 → 발주 확정(ACTIVE) → 추천 데이터 갱신이다. 제품 확인처럼 즉시 답이 필요한 요청은 REST로 처리하고, 결제 후 이어지는 상태 변경은 Kafka 이벤트로 전달한다.

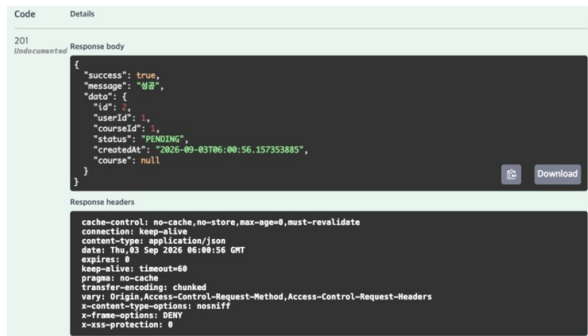
기존 구조의 payment.completed는 결제 완료를, enrollment.completed는 등록 완료를 뜻한다.

MediBridge에서는 후자를 발주 확정에 대응시키되, 실제 연동 전에는 이벤트 이름과 필드의 의미를 팀이 함께 확정해야 한다.

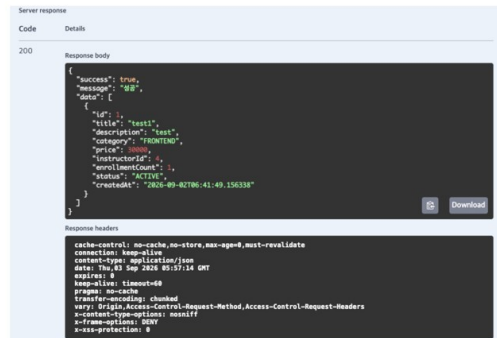
000000 201



000000 201 PENDING



000000 200



000000 200 ACTIVE

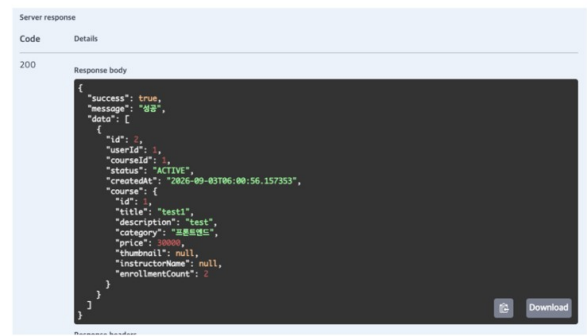


그림 2 실습 API 응답 화면과 상태 변화

응답 구분	HTTP	화면에서 확인한 값	해석
계정 생성 응답	201	role: STUDENT	실습 계정 생성과 역할 필드를 확인
목록 조회	200	category, price, ACTIVE	목록과 상태 응답 구조를 확인
등록 생성	201	PENDING	접수 직후 대기 상태를 확인
등록 조회	200	ACTIVE	후속 조회의 상태 변경을 확인

화면에는 STUDENT, courseId 등 강의 실습용 값이 남아 있다. 따라서 이 자료는 기본 API 흐름의 근거로 활용하며, 병원·약국 권한과 실제 의약품 데이터 검증까지 끝났다는 의미로 해석하지 않는다. 결제 이벤트로 상태가 바뀌는 과정과 추천 갱신은 별도로 확인해야 한다.

4 트러블슈팅과 학습 내용

4.1 실습에서 확인한 문제

문제	원인과 조치	다음 작업에 적용할 점
pytest에서 api 모듈을 찾지 못함	패키지 구조와 실행 위치를 확인하고 <code>__init__.py</code> 를 추가했다.	오류 파일뿐 아니라 import 경로와 실행 위치를 함께 확인한다.
설치한 라이브러리를 불러오지 못함	설치 환경과 실행 환경이 달랐다. 가상환경의 Python으로 설치와 실행을 맞췄다.	<code>python -m pip</code> 방식으로 같은 환경에 설치한다.
MariaDB 컨테이너에 Workbench 연결 실패	호스트에 공개한 포트와 접속 포트가 달랐다. 포트 매핑에 맞춰 컨테이너 설정을 수정했다.	호스트 포트와 컨테이너 내부 포트를 구분한다.
Mac에서 실행 시 모듈 오류	python과 python3가 가리키는 환경이 달랐다. 필요한 패키지가 있는 Python으로 실행했다.	실행 명령과 가상환경 경로를 README에 남긴다.
테스트의 done 검증 실패	구현에 없는 필드를 검사하고 있었다. 현재 계약의 id와 status를 기준으로 수정했다.	요구사항과 API 계약을 먼저 대조한 뒤 실제 값과 상태를 검사한다.

4.2 기술을 연결하며 이해한 점

Eureka는 서비스 위치를 찾고 Gateway는 요청을 전달한다. Swagger UI는 API 형식을 살펴보고 요청을 직접 보내 보는 도구이며, Pydantic은 Python API의 입력과 출력 형식을 정의하는 데 활용한다. 각 도구의 역할을 구분하면 문제가 난 구간을 더 빠르게 찾을 수 있다.

4.3 다음 단계의 확인 항목

결제 완료 메시지를 두 번 받아도 발주와 구매 이력이 중복 반영되지 않아야 한다. 처리한 이벤트 식별자를 기록하고, 상태 변경과 처리 기록이 함께 저장되도록 구현하는 방안을 검토한다. 결제 기록은 저장됐지만 이벤트 전송이 실패하는 경우도 재현해 복구 방법을 정한다.

추천은 신규 거래처, 후보 0건, 판매 중단 품목, 잘못된 제품 분류를 확인한다. 추천 서비스가 중단되어도 제품 조회와 발주가 가능한지 점검하고, 제약사 데이터 형식이 바뀌면 기존 데이터와 새 데이터 모두 검증한다.

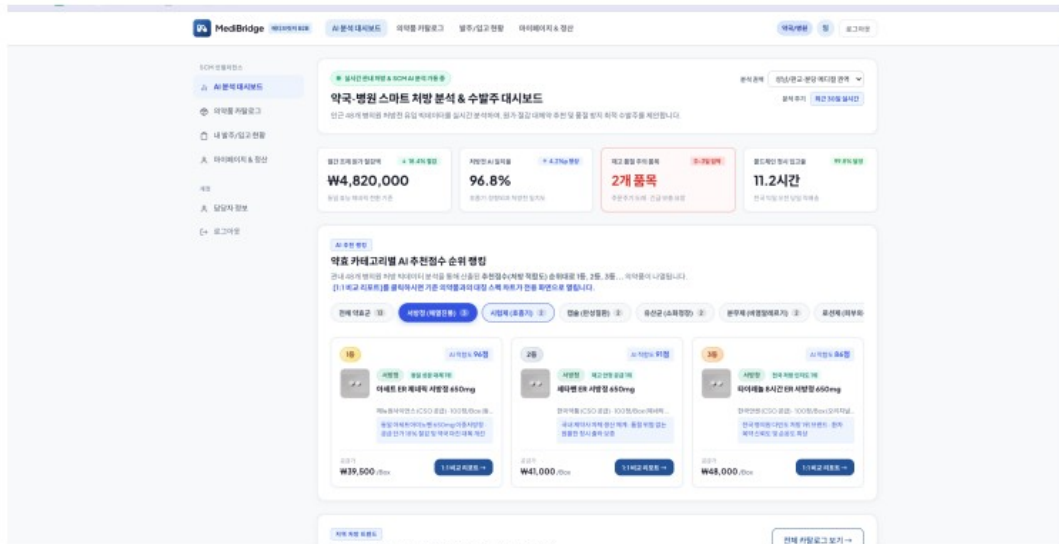
원문 자료에서 스프린트 1은 완료, 결제·Kafka·추천 중심의 스프린트 2는 예정으로 구분되어 있다. 화면 자료와 별개로 기능의 연동 완료 여부는 테스트 결과를 통해 판단한다.

5 화면 구성과 업무 적용

5.1 제품 탐색과 발주 화면

5. MVP

메인 사이트



제품 주문



그림 3 조별 과제의 메인 화면과 제품 주문 화면

메인 화면에서 후보 품목을 살펴보고 제품 상세 화면에서 발주를 진행하는 구성이다. 검토 시에는 가격과 제품 정보가 잘 보이는지, 발주 후 상태를 바로 확인할 수 있는지를 중심으로 살펴본다.

5 화면 구성과 추천 표현

5.2 제품 등록과 추천 화면

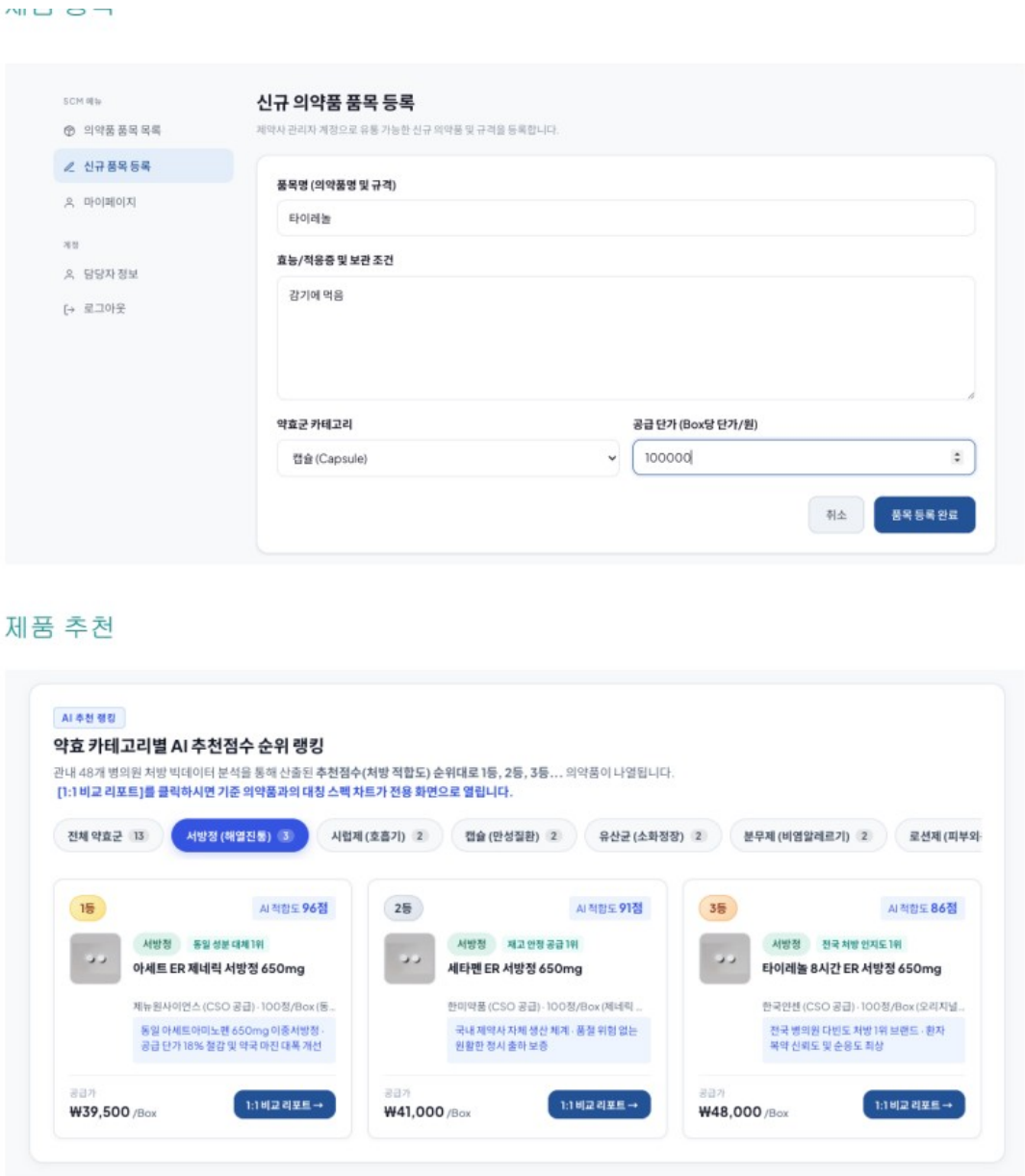


그림 4 조별 과제의 제품 등록과 추천 화면

제품 등록 항목은 제약사별 자료를 같은 기준으로 받는 출발점이다. 추천 화면의 점수와 성과 수치는 제시된 화면의 표현이며, 검증된 정확도로 해석하지 않는다. 초기 버전에서는 구매 약효군이나 인기 품목 등 실제 사용한 규칙을 추천 이유로 보여 주는 것이 적절하다.

6 사용 사례와 기대 효과

다음은 MediBridge에서 검증할 수 있는 활용 예시이다. 사용 사례를 통해 장점을 구체화하고, 실제 효과는 사용 기록과 담당자의 의견으로 확인한다.

사용 상황	적용 방법	장점과 확인 지표
관련 품목을 비교하는 약국	자주 구매한 약효군에서 미구매 품목을 최대 5개 보여 준다.	찾아볼 대상을 줄인다. 탐색 시간, 추천 클릭률, 발주 전환율을 확인한다.
처음 가입한 거래처	구매 이력이 쌓이기 전에는 인기 품목과 선정 기준을 안내한다.	시작 화면을 비워 두지 않는다. 첫 발주까지 걸린 시간과 추천 거절 사유를 본다.
상담을 준비하는 CSO	거래처 구매 이력과 추천 이유를 함께 확인한다.	개인 경험에만 의존하지 않고 상담을 준비한다. 준비 시간과 제안 수용 여부를 확인한다.
제품 자료를 갱신하는 제약사	product-service에서 공통 필드로 변환하고 누락 값을 검증한다.	수정 범위를 줄인다. 오류 비율과 정보 반영 시간을 측정한다.
결제 후 상태를 확인하는 운영자	완료 이벤트로 발주를 확정하고 추천용 구매 이력을 갱신한다.	반복 대조 작업을 줄인다. 처리 지연과 상태 불일치 건수를 확인한다.

6.1 장점을 유지하기 위한 운영 조건

추천을 쉽게 설명할 수 있어도 제품 정보가 오래되면 신뢰를 얻기 어렵다. 정보 갱신 시점을 관리하고, 판매 중단 품목은 추천에서 제외해야 한다. 제품 분류가 잘못된 경우에는 추천 규칙보다 먼저 원천 데이터를 수정한다.

독립적인 서비스는 변경 범위를 줄이는 데 도움이 되지만 운영 비용도 늘어난다. 추천 개선 빈도와 장애 영향을 살펴보고, 필요할 때 해당 서비스의 실행 자원만 늘린다. 경영 관점에서는 투자 우선순위를, 업무 관점에서는 처리 시간을, 기술 관점에서는 오류와 복구 시간을 함께 확인한다.

6.2 이후 확장 방향

재주문 시점을 제안하려면 미구매 후보 추천과 별도로 반복 구매 품목, 구매 주기, 재고 정보가 필요하다. 초기 추천이 실제 발주에 도움이 되는지 먼저 확인한 뒤 필요한 데이터를 모아 확장한다. 유사 거래처 추천도 거래처 유형과 데이터 품질이 충분할 때 검토할 수 있다.

7 실제 현업 사례와 적용 아이디어

7.1 Amazon Pharmacy의 수요 계획

AWS 공식 사례에서 Amazon Pharmacy는 판매 이력과 현재 주문 데이터를 활용해 일별 수요 계획을 개선했다. 수기 계획 작업을 주당 약 5시간 줄였다고 소개한다. 기존 업무에 예측 기능을 연결해 반복 작업을 줄인 사례이다.

찾아본 기능은 수요 예측과 최신 데이터 반영이다. MediBridge에서도 발주 이력을 먼저 정확히 모으고, 이후 반복 구매 품목의 주문 시점을 제안하는 데 적용해 볼 수 있다. 초기에는 예측 성능보다 담당자의 확인 시간이 줄었는지부터 살펴보면 좋겠다.

AWS 고객 사례 <https://aws.amazon.com/solutions/case-studies/amazon-pharmacy-case-study/>

7.2 Merck Life Science의 공급망 데이터 통합

Google Cloud 사례에서 Merck Life Science는 BigQuery로 데이터를 통합하고 Vertex AI로 공급망 수요에 대응하는 모델을 구축했다. 기존에는 ERP 데이터 이동과 변환에 약 이틀이 걸렸으나, 통합 후에는 더 최신의 데이터를 활용할 수 있게 되었다.

찾아본 기능은 서로 다른 업무 데이터를 연결하는 과정이다. MediBridge에서도 제품과 발주 데이터의 식별자 및 갱신 기준을 맞추는 데 적용해 볼 수 있다. 추천 모델을 고도화하기 전에 데이터가 제때 반영되는지 확인하는 것이 먼저라고 판단했다.

Google Cloud 고객 사례 <https://cloud.google.com/customers/mercklifescience>

7.3 Manipal Hospitals의 의약품 주문 흐름

Manipal Hospitals의 ePharmacy는 처방 확인부터 주문, 수령 방식 선택까지 연결한다. Google Cloud 공식 사례에서는 주문 접수 시간이 15분에서 5분 미만으로 줄였다고 소개한다. API를 통해 데이터를 연결해 사용자 업무를 단순하게 만든 사례이다.

찾아본 기능은 여러 단계를 하나의 주문 흐름으로 잇는 방식이다. MediBridge에서도 제품을 찾고 발주 상태를 확인하기까지의 시간을 측정해 볼 수 있다. 개별 API의 성공 여부와 함께 사용자가 중간에 멈추는 지점을 보면 개선 순서를 정하기 쉬울 것 같다.

Google Cloud 고객 사례 <https://cloud.google.com/customers/manipal-hospitals>

위 결과는 각 기업의 공개 사례이며 MediBridge의 성과는 아니다. 적용 아이디어는 업무를 줄이는 방식과 데이터 연결 과정을 참고해 정리했다.

8 참고자료와 구현에 적용할 점

8.1 Kafka 이벤트 연동

우아한형제들 기술블로그의 「우리 팀은 카프카를 어떻게 사용하고 있을까」에서 주문·배달 이벤트 연결과 전송 누락에 대응하는 Outbox 방식을 찾아보았다. MediBridge에서도 결제 저장 후 이벤트가 전달되지 않는 상황을 점검하는 데 참고할 수 있다. 우선 발행과 수신 흐름을 확인하고, 재처리 시 중복 반영을 막는 기준을 함께 정해 보 고자 한다.

우아한형제들 기술블로그 <https://techblog.woowahan.com/17386/>

8.2 Eureka 서비스 등록과 조회

Spring의 Service Registration and Discovery 가이드에서 서버에 서비스를 등록하고 이름으로 찾아 호출하는 과정을 확인했다. product-service의 주소를 직접 입력하는 대신 이름으로 찾도록 적용해 볼 수 있다. 서비스를 재시작한 뒤 다시 연결되는지 확인하는 실습에 활용하기 좋다.

Spring 개발 가이드 <https://spring.io/guides/gs/service-registration-and-discovery/>

8.3 API 문서와 요청 확인

FastAPI의 First Steps에서 API를 만들고 /docs의 Swagger UI로 요청을 보내는 과정을 확인했다. recommend-service의 입력값과 추천 결과 형식을 팀과 공유하는 데 적용할 수 있다. 구매 이력이 있는 경우와 없는 경우를 같은 화면에서 시험해 보면 결과 차이를 이해하기 쉽다.

FastAPI 개발 가이드 <https://fastapi.tiangolo.com/tutorial/first-steps/>

8.4 Docker 실행 환경 구성

Docker Compose Quickstart에서 여러 서비스를 설정 파일로 정의하고 함께 실행하는 과정을 찾아보았다. MediBridge의 API와 데이터베이스를 같은 실행 절차로 준비하는 데 적용할 수 있다. 포트와 환경 변수도 함께 기록하면 실습에서 겪은 연결 오류를 줄이는 데 도움이 될 것 같다.

Docker 시작 가이드 <https://docs.docker.com/compose/gettingstarted/>

8.5 스프린트 검토와 개선

Atlassian의 Sprint Review 설명에서 결과를 이해관계자와 검토하고 다음 작업을 조정하는 과정을 확인했다. 추천 화면을 보여 준 뒤 거절 사유를 기록하고 작업 목록에 반영하는 데 적용해 볼 수 있다. 단순한 진행 보고보다 실제 사용 의견이 남는 검토가 필요하다고 보았다.

Atlassian Agile 가이드 <https://www.atlassian.com/agile/scrum/sprint-reviews>

9 종합 결론

MediBridge의 우선 과제는 구매 담당자가 제품을 찾고 발주하는 일을 더 쉽게 만드는 것이다. 기존 업무에 설명 가능한 추천을 연결하고, 사용자가 실제로 도움을 받는지 작은 범위에서 확인하는 방향이 적절하다.

Agile은 현장 의견을 다음 개발 주기에 반영하는 데 활용한다. MSA는 제품, 발주, 결제, 추천의 관리 책임을 나누어 수정 범위를 줄이는 데 활용한다. 두 방식 모두 도입 자체보다 사용자 불편과 운영 부담을 얼마나 줄였는지로 평가해야 한다.

실습을 통해 실행 환경, 데이터 형식, API 계약이 서로 맞아야 기능이 연결된다는 점을 확인했다. 다음 단계에서는 결제 이벤트와 추천 갱신을 검증하고, 신규 거래처와 추천 결과가 없는 상황까지 점검한다. 이후 사용 기록과 피드백을 바탕으로 재주문 제안 등 필요한 기능을 추가한다.

개발과 검증의 우선순위

순서	실행할 작업	확인할 결과
1	실습용 필드와 권한을 실제 거래처 및 의약품 업무에 맞춘다.	사용자 역할, 제품 분류, 발주 상태의 의미가 일치한다.
2	결제 완료와 발주 확정, 추천 갱신을 연결한다.	정상 처리와 실패 후 재처리에서 데이터가 일관된다.
3	구매 담당자에게 화면을 보여 주고 의견을 기록한다.	탐색 시간과 추천 거절 사유를 파악한다.
4	사용성과 운영 부담을 비교해 다음 범위를 정한다.	필요한 기능에 개발 시간과 운영 자원을 배분한다.

작성에 활용한 프로젝트 자료

기존 개인 보고서에 수록된 조별 과제의 프로젝트 개요, 이해관계자, 백로그, 아키텍처와 화면 자료를 활용했다. 실습 API 응답은 기본 흐름과 상태값을 설명하는 근거로 정리했다. 외부 기술 자료와 현업 사례는 해당 설명 바로 아래 링크를 표시했다.

외부 링크 확인일 2026년 9월 7일